

1.1 模型

1.1.1 个性化定制任务

(1) 记可重构调度车间共有 n 个 CPP 任务，每个任务 $J = \{J_1, J_2, \dots, J_n\}$ 包含两个变量 $J_i = \{operation_i, seq_i\}$ 。

(2) 每个任务 i 包含 n_i 个操作, $n_i \in \{o_1, \dots, o_k\}$ 。为了简化调度模型中参数表达，本文使用 j 替代 n_i 。因此，变量 X_{ij} 表示任务 i 的操作 j 的 X 变量的值。每个任务包括两个变量 OPT_{ij} 和 q_{ij} ，分别表示加工阶段的类型和工作量。

(3) seq_i 是 $n_i \times n_i$ 矩阵，用于表示加工阶段之间的时间约束，每个矩阵元素的值 x_{mn}^i 定义为：

$$X_{mn}^i = \begin{cases} 1 & \text{if operation } n \text{ is the next step of job } m \\ 0 & \text{otherwise} \end{cases} \quad (4-1)$$

1.1.2 可重配置机器

(1) 记一个车间共有 R 台可重构机器，每个机器 $M = \{m_1, m_2, \dots, m_R\}$ 包括七个变量 $m_r = \{MATY_r, Pt_r, Rt_r, Tt_r, Sp_r, Ast_r, Aet_r\}$

(2) $MATY_r$ 是机器可加工阶段的集合， Rt_r 是 $\|MATY_r\| \times \|MATY_r\|$ 维矩阵，表示不同可加工阶段之间的重配置时间。 Pt_r 是 $\|MATY_r\|$ 维向量，表示单位工作量下的任务加工时间。

(3) Tt_r 是 $\|r + 2\| \times \|r + 2\|$ 矩阵，表示车间中原材料存储和每个重构机器之间的原材料转运时间，每个半成品向下一个重构机器的转运时间以及加工成品向成品存储点的转运时间。

(4) 对于每一个机器 M_r ， Ast_r 和 Aet_r 表示加工过程中每个加工任务可以开始和结束的加工时间。 Sp_r 表示每个加工阶段的加工时间。

1.1.3 资源重配置过程

(1) 阶段调度矩阵 σ_i 是任务 i 的最终操作加工次序，也是调度模型中的优化变量。 σ_i 是一个 $n_i \times R$ 维的矩阵，矩阵上的元素值 β_{jr}^i 定义为：

$$\beta_{jr}^i = \begin{cases} 1 & \text{if operation } j \text{ of job } i \text{ is to be excuted in machine } r \\ 0 & \text{otherwise} \end{cases} \quad (4-2)$$

(2) 任务阶段 J_{ij} 的调度时间包括开始时间和结束时间两个变量，分别记为 Sot_{ij} 和 Eot_{ij} ，故每个任务阶段的实际加工时间为 $Eot_{ij} - Sot_{ij}$

(3) 所有任务的完工时间(makespan)记作 $\max_{i \in 1 \dots n} \{T_i\}$ ，其中 T_i 表示任务 i 最终抵达成品存储点的时刻。

1.1.4 动态重配置调度模型

为了实现最优的动态重构调度策略，本文通过运筹模型将调度矩阵、完工时间和动态重构因素之间的关系进行建模，具体模型如下所示。

$$\min \max_{i \in 1 \dots n} \{T_i\} \quad (4-3)$$

批注 [铨罗1]: 未作出明确定义，似乎应该将其解释为机器的实际加工速度

批注 [铨罗2]: 似乎应该将 r 改为 R

$$s.t. OPT_{ij} \in \cup_{r=1}^S \beta_{jr}^i \times MAT_{Y_r} \quad (4-4)$$

$$Ast_r \geq \sum_{r=1}^S \beta_{jr}^i \times Tt_{0r} \quad (4-5)$$

$$x_{jp}^i \times (Sot_{ip} - Eot_{ij} - \sum_{u=1}^S \sum_{v=1}^S \beta_{ju}^i \beta_{pv}^i Tt_{uv}) \geq 0 \quad (4-6)$$

$$Eot_{ij} - Sot_{ij} = \sum_{r=1}^S \beta_{jr}^i \times q_{ij} \times Pt_r(OPT_{ij}) \times Sp_r(OPT_{ij}) \quad (4-7)$$

$$T_i = Eot_{ij} + \sum_{r=1}^S \beta_{jr}^i \times Tt_{rs+1} \quad (4-8)$$

$$\beta_{jr}^i \beta_{pr}^l (Sot_{ip} - Eot_{ij})(Sot_{ip} - Eot_{ij} - \sum_{u=1}^S \sum_{v=1}^S \beta_{ju}^i \beta_{pv}^i Rt_{uv}) \geq 0 \quad (4-9)$$

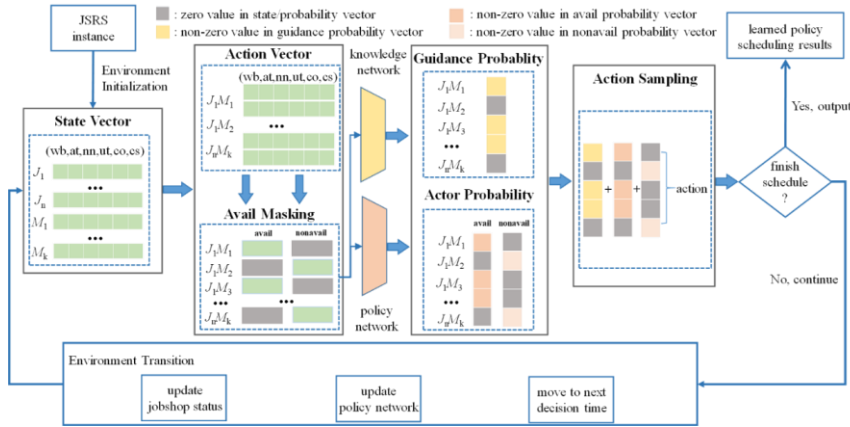
$$\beta_{jr}^i \times (Sot_{ij} - Ast_r) \geq 0 \quad (4-10)$$

$$\beta_{jr}^i \times (Eot_{ij} - Aet_r) \leq 0 \quad (4-11)$$

$$i \in 1 \dots n, j \in 1 \dots k, r \in 1 \dots R \quad (4-12)$$

$$l \in 1 \dots n, p \in 1 \dots k \quad (4-13)$$

约束(4-4)确保每个阶段都被分配到可加工的机器上。约束(4-5)保证每个任务的第一个阶段都在原材料转运后开始加工。约束(4-6)让先前的加工阶段和半成品的转运都在下一个阶段开始加工前完成。约束(4-7)表明了每个任务阶段的加工时间。约束(4-8)表明每个任务的完工时间是当任务制造产品被转移至完成品存储区域时。约束(4-9)强调了动态重配置时间的重要性。下一个任务阶段的开始加工时间是上一个任务加工完成后加上两个阶段进行资源重配置所需要的时间。约束(4-10)和(4-11)确保加工时间处于机器的工作范围。注释(4-12)和(4-13)定义了上述所有约束中变量取值范围。



批注 [钩罗3]: 未对 s 做出定义, 若元素 s 属于集合 S, 则 S 应是可用于承担 j 任务加工流程的可重构机器集合

1.2 算法

在这个章节，提出了基于知识引导的深度强化学习算法(Knowledge-Guided Deep Reinforcement Learning, KGDL)，算法的具体运行机制如上图所示。动态重构调度过程实际上由一系列调度决策组成，这些决策用于将当前的加工任务分配到对应的可重构机器中，智能体在决策的每一步做出的决策都是变量 $\beta_{j,r}^i$ 在每个时刻的取值。在进行调度的过程中，首先需要将制造车间初始化成状态变量，其中每个机器和每个任务都根据特征输出函数进行状态表征。由于调度决策实际上由机器任务对组成，因此任务向量和机器向量被合并到一起，形成动作向量，由于后续的动作决策。在通过基于不可用动作掩码之后，知识引导网络和动作网络将分别计算动作概率，最后智能体将根据合并的动作概率进行抽样决策，选取当前状态下的最优调度动作。在完成决策后，智能体将判断车间环境是否完成全部任务的生产加工，如果没有，则继续迭代，当有新的任务完成后开始继续当前循环决策，直至所有任务完成加工。

1.2.1 马尔可夫决策过程

马尔可夫决策过程(Markov Decision Process, MDP)由五个关键变量组成，分别为状态，动作，迁移，奖励，策略，本文将从这五个变量详细解释车间动态调度的MDP过程。

1.2.1.1 状态向量

状态向量由于表征车间系统并引导调度决策。在 t 时刻的状态向量被记为 $S_t = \{N_t(i), i \in \{J, M\}\}$ ，这一状态向量包括所有机器和任务节点的状态向量。为了全面刻画车间场景的特征，每个节点包含六个特征，特征函数为 $N_t(i) = (wb_t, at_t, nn_t, ut_t, co_t, cs_t)_i, \forall i \in \{J, M\}$ 。每个特征的具体解释如下所示。

特征名	释义
wb	节点的工作状态(0 记为空闲，1 记为工作)
at	节点的空闲时间
nn	可以到达的节点数量
ut	节点的时间利用率(工作时间/总时长)
co	当前节点的阶段数
cs	当前节点的加工速度

1.2.1.2 动作向量

每个动态调度动作其实是一个任务动作对，因此动作向量其实是阶段选择和机器分配的联合动作。考虑到机器重构的有限性，部分任务在完全加工时其下一阶段并不能被当前所有的空闲机器进行加工，因此存在在特定时间并没有可用的任务动作对的可能，所以在时间 t 的动作向量被定义为 $a_t = \{(o_{ij}, M_k)\} \cup \{None\}$ ，其中 O_{ij} 代表可加工任务阶段，而 M_k 代表空闲且可加工的机器。None 表示当前状

态智能体不采取任何加工动作。动作向量由状态向量聚合而成 $S'_t = \text{concat}(N_t(J), N_t(M))$ 。在动态可重构调度问题中, 有时机器选择 **None** 动作, 能够使其在后续选择一个需要更短加工时间的动作, 最终可以实现更短的动态重构调度时间。换言之, 不可用动作也存在长期视角的价值, 不可被完全忽略, 所以当我们在使用这个动作向量进行策略输出时, 将动作向量进行了额外的不可用动作掩码的操作, 并分别输出了可用动作和不可用动作的动作选择概率, 让智能体独立的学习采用 **None** 动作的潜在价值。

1.2.1.3 迁移向量

状态向量取决于先前的状态和动作, 因为随着任务的加工和机器状态的动态变量, 其对应的特征值会发生变化, 状态向量也会发生改变。所以状态的迁移函数为 $S_{t+1} \leftarrow S_t | (a_t = a)$, 当 S_t 完成调度决策后, 如果有新的加工任务完成, 则状态迁移到新的状态向量 S_{t+1} 。

1.2.1.4 奖励函数

奖励函数被用于计算最优动作并评估策略的有效性。在现有的关于车间调度的文献中, 将奖励函数的设计为 S_t 和 S_{t+1} 的预期结束加工时间的差异。将这一奖励函数进行累加就能得到整个车间的预期完工时间的负值, 因此该奖励函数能实现最优的动态重构调度时间。本文也沿用了这一计算方法。另外本文还考虑了机器重配置时间的对加工整体进程的影响, 指导智能体学习最短动态重配置时间的调度策略。因此动态重配置调度问题的奖励函数为 $r(S_t, S_{t+1}, a_t) = T(S_t) - T(S_{t+1}) - \text{trantime}(a_t)$, 其中 $T(S)$ 是当前状态 S 的时刻, 而 trantime 是可用重配置机器的重配置时间。

1.2.1.5 策略

基于当前的调度策略 $\pi(a_t | S_t)$ 被定义为每个状态下对于当前动作集的概率分布。本文在每个调度时间, 基于当前状态向量分别输出了可用动作和不可用动作的调度策略。为了更快地学习有效的策略, 我们在输出动作概率时部署了知识引导网络。知识引导网络也输出动作的概率分布, 而智能体将根据知识引导概率分布, 可用动作概率分布, 不可用动作概率分布三部分合并的动作概率向量进行抽样, 以获取最优的调度策略。

1.2.2 知识引导网络

车间调度问题是强 **NP-hard** 问题, 而机器重配置变量的引入使这个问题的求解时间进一步复杂化。因此, 在传统的深度强化学习算法中, 本文部署了一个知识引导结构来解决这个问题。知识引导网络在增强训练结果的有效性和加快训练速度方面的有效性已得到广泛验证。**if-then** 规则是一种流行的知识形式, 基于这一逻辑, 采用规则方法的知识引导框架已被验证, 可以改善强化学习算法训练结果并加速算法训练过程。

为了最大限度地减少 **makespan**, 本文根据提出了一种通用的启发式知识引导框架来加快智能体的快速策略迭代。这种启发式方法受到 **IF A THEN B** 结构和 **e-greedy** 策略原则的影响。先验知识的核心逻辑可以总结如下: 如果对某个可用机器进行任务分配时, 其所需要的重新配置时间更短, 并且其后续的任务工

作量总和更重，那么选择这个动作的经验引导概率更高。这种方法在引导智能体的学习过程中实现了探索和利用的平衡，防止强化学习系统陷入局部最优解。考虑到重置时间为零的可能性，可用动作的知识引导概率分布为：

$$S_1 = \text{trantime}(a_t) / \|\text{trantime}(a_t)\|_2, S_2 = \text{workload}(a_t) / \|\text{workload}(a_t)\|_2 \quad (4-14)$$

$$p_{a_t} = \alpha_1 (e^{-S_1} / \|e^{-S_1}\|_1) + \alpha_2 (e^{S_2} / \|e^{S_2}\|_1)$$

其中trantime表示可用动作的机器重配置时间，workload表示可用动作剩余的任务处理时间。不可用动作的知识引导概率为零向量，因此知识网络输出单个概率向量，而不是像在动作网络中输出两个概率向量。综上所述，知识引导网络本质上作为动作概率向量的输出函数。由于不同重构任务之间的重配置时间和加工处理时间差异很大，使用像 softmax 这样的标准激活函数可能会导致概率分布过于集中（0-1 分布）或过于分散（均匀分布）而无法有效。因此我们在知识引导网络和动作网络都采用 1 范数的激活函数，以取得更平稳的动作概率分布。

1.2.3 算法部署

1.2.3.1 训练流程

KGDRL 算法使用近端优化策略(Proximal Policy Optimization, PPO)结构进行训练，并部署了传统深度强化学习中的动作-评论网络(Actor-Critic)用于策略评估和迭代。算法通过逐步优化策略有效防止在高维动作空间中出现过大的更新，从而避免了策略震荡或不稳定的训练，增强算法对可重配置加工任务动作空间的探索能力，确保算法在个性化定制任务加工流程中高效迭代加工任务。动作网络用于生成策略 π_ω ，评论网络用于策略评估 V_ϕ 。 π_ω 和 V_ϕ 都采取传统的深度学习中的网络设计方法，均为两层深度 Q 网络并含有一层隐藏层的形式，具体的算法训练流程如[错误!未找到引用源。](#)所示，训练被设置 I 次迭代和 β 个独立重构调度算例进行，在每次迭代期间，算法计算每种状态的知识引导概率，并将其纳入动作网络输出的概率中，以优化智能体的采样策略。可用动作的动作概率向量是使用可用动作的状态向量计算的，不可用动作也是如此。

KGDRL 算法中的超参数在 $5*5$ 的算例上根据最优调度进行了调整，并在其他算例维度采用相同的超参数设置。为了获得最佳的训练效果，训练期间的迭代次数和实例批量大小分别被设置为 $I = 1000$ 和 $\beta = 10$ ，算法的最小和最大重放缓冲区分别为 128 和 2048，使得智能体能在更短的时间内学习到有效的调度策略。对于 PPO 损失，裁剪比和熵系数分别为 0.1 和 0.01，以保证调度策略的收敛性。PPO 损失的迭代训练数被设置为 15。为了表格整洁将 PPO 损失的计算方式及更新步骤单独说明，具体步骤如下：

- 1.计算 $\delta_t = r_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$

- 2.计算 GAE $A_t = \sum_{i=0}^{\infty} (\gamma\lambda)^i \delta_{t+i}$ ，以及目标值 $TG_t = A_t + V_\phi(S_t)$

- 3.计算 PPO 损失 $\Delta_{clip} = E_t[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$ ，并计算 $\Delta_{value} = E[(V_\phi(S_t) - TG_t)^2]$

- 4.更新参数 $\omega = \omega + \sum_t \delta_t \Delta_{clip} \log \pi_\omega(a_t | S_t)$ 和 $\phi = \phi + \sum_t \delta_t \Delta_{value} V_\phi(S_t)$

算法 1 : KGDRL

初始化策略网络 π_ω 和评估网络 V_ϕ ，设置可训练参数 ω 和 ϕ

初始化 β 个独立的车间调度算例

当 $iter = 1, 2, \dots, I$ 时运行:

 当 $b = 1, 2, \dots, \beta$ 时运行:

 基于 b ，**初始化** S_t

 当 S_t 没有终止时运行:

 基于 S_t ，初始化 a_t ，掩码向量 F_{avail} 和 $F_{nonavail}$

$F_{avail} = 1, F_{nonavail} = 0$ if $O_{ij} \in a_t$

 基于策略网络计算动作分布概率

$$p_{avail} = \pi_\omega(F_{avail}(O_{ij})S_t), p_{nonavail} = \pi_\omega(F_{nonavail}(O_{ij})S_t)$$

 计算知识引导动作概率分布 $p_{advice} = p_{a_t}$

 基于联合动作概率 $p = p_{advice} + p_{avail} + p_{nonavail}$ 抽样最终决策动作 a

 计算奖励 r_t 和下一状态向量 S_{t+1}

$$S_t \leftarrow S_{t+1}$$

 对于每一步，基于 r_t 和 S_{t+1} 计算广义优势估计值(GAE) A_t

 基于 A_t 计算 PPO 损失 Δ

 基于 PPO 损失 Δ 更新网络参数 ω 和 ϕ

Return

1.2.3.2 算例配置

与传统的调度问题不同，在 CPP 生产问题中，机器重新配置时间与任务阶段相关，但与任务序列。因此无法利用公共数据集来验证 KGDRL 算法的有效性，进。为了确保 KGDRL 方法在所有 CPP 问题中的有效性，本文构建动态重构调度算例用于算法训练。在动态重构车间调度问题中，任务阶段加工时间 $p_{ij} =$

$q_{ij} \times Pt_r(OPTY_{ij})$ ，机器重配置时间和任务阶段加工速度是最为重要的三个变量，在生产独立算例通过随机生成这三个变量参数确保算例具有实际车间调度算例的代表性。同时为了确保算法在不同复杂度下均能获得良好的调度结果，本文在六个维度上通过正态分布随机生成不同加工时间、重配置时间和处理速度的调度算例。在大部分实际制造场景中，机器重配置时间小于任务阶段加工时间，因此，本文将平均重新配置时间设置为平均任务阶段加工时间的 0.1 倍，将平均机器处理速度设置为 1，具体的参数设置如错误!未找到引用源。所示。

算例维度 (任务数*机器数)	P_{ij}	Rt_r	Sp_r
5*5, 10*5, 20*5, 5*10, 10*10, 20*10	N(100,0.1)	N(10,0,1)	N(1,0.1)

1.3 实验评估与验证

1.3.1 基准算法选取

为了更全面准确的评估 KGDRL 算法在动态高维调度场景上的表现, 将通过训练 KGDRL 学习到的调度策略与三种在。车间调度问题中广泛使用的启发式规则和三种著名的 DRL 算法进行了比较。

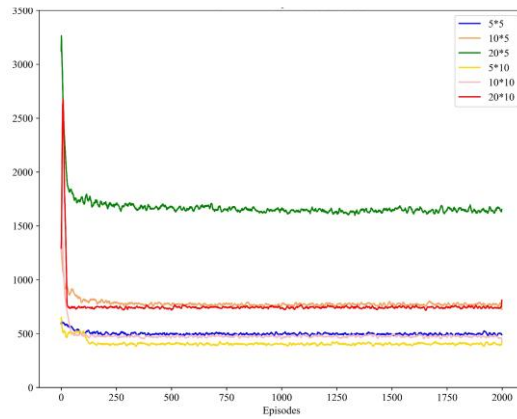
- Random: 随机选择任意任务进行加工。
- MWKR(Most WorK Remain): 选择剩余工作最多的任务进行加工。
- FIFO(First in First Out): 选择首先到达待加工区的任务进行加工。
- DDQN(Double Deep Q Network): 双深度 Q 网络算法。
- A2C(Advantage Actor-Critic): 基于优势函数的动作-评论网络算法。
- DDPG(Deep Deterministic Policy Gradient): 深度确定性策略梯度算法。

本文从收敛性, 稳健性, 策略有效性的角度将 KGDRL 算法和这些基准对比, 全面评估论证 KGDRL 算法在解决面向 CPP 问题中求解动态重构调度生产问题的性能。

1.3.2 收敛性分析

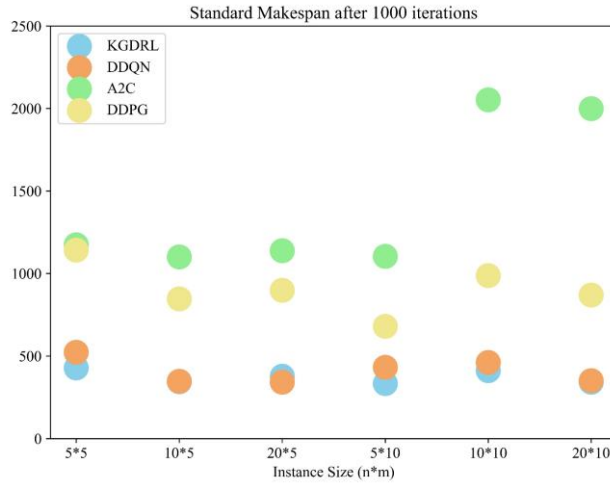
1.3.2.1 算法训练的收敛性评估

KGDRL 方法的训练过程稳定且收敛迅速。如下图所示, 六种不同大小的训练曲线在前 200 次迭代训练中出现一些波动, 然后迅速收敛。借助知识引导网络, KGDRL 智能体可以在调度过程中找到有效的调度动作, 并快速收敛到稳定的调度策略。并且算法的收敛性能在所有维度的算例上都表现出了一致的趋势, 这凸显了所提出的知识引导机制的必要性。



KGDRL 训练图

在基线算法中，启发式规则不存策略学习问题阶段，因此没有训练阶段的收敛性的评定，而对于其他三个强化学习基线算法，收敛性则是评估其算法能力的重要指标。为了进一步揭示 KGDRL 强化学习策略的收敛性，本节计算了所有算法在每个算例上迭代训练 1000 次后的标准完工时间(standard makespan)进行比较。具体来说，我们通过计算 1000 次迭代后的标准完工时间来评估所有基线算法的收敛性。标准完工时间的计算方法是将每个实例的完工时间除以算例参数（定义为 $[n/m]$ ，其中 n 是 CPP 任务， m 是可重构机器数）。例如，对于 $5*10$ 的调度算例，其算例参数为 1。标准完工时间的结果下图所示。可以看到，对于收敛的调度策略，在 1000 次迭代训练后的调度策略在各个维度上的标准完工时间都趋向于收敛的固定值。具体来说，KGDRL 和 DDQN 两个算法表现出稳定的策略，它们在所有六个算例上的标准完工时间始终接近 400。相比之下，其他两个基线强化学习算法，A2C 和 DDPG 则没有表现出收敛性，不仅在大多数算例上的标准完工时间超过 800，并且其完工时间随着算例维度的增加而显著增加，表明其不具备有效的收敛性。在这两种算法中观察到的高训练波动表明它们的训练结果不可靠，A2C 和 DDPG 在训练过程会受到参数变化的影响而无法在动态场景中搜索最优价值动作。因此，由于 A2C 和 DDPG 提供的结果没有说服力，我们选择 DDQN 算法和其他三个规则作为基线算法与 KGDRL 进行不同算例上的调度结果比较。

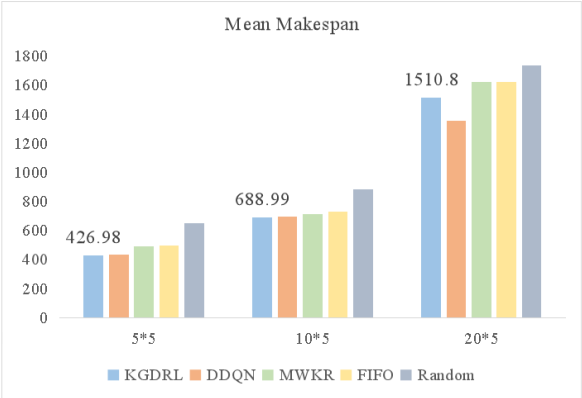


各算法的标准完工时间

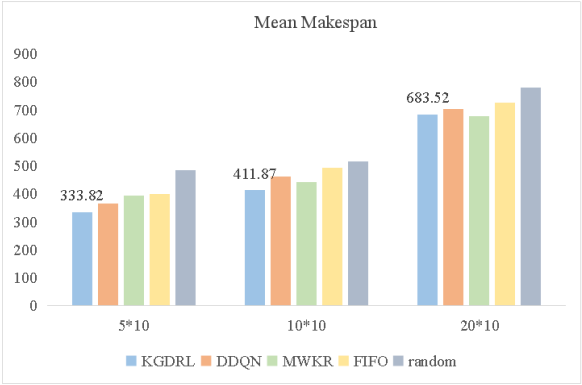
1.3.2.2 算法在不同算例维度上的收敛结果分析

基于上节分析，A2C 和 DDPG 在训练阶段难以获得收敛的调度策略，因此其训练结果不具备参考价值。所以在多维度比较实验中，本文只考虑 DDQN 算法和其他三个调度规则的调度结果进行比较，用于分析其收敛调度策略是否能在多维度算例上均获得表现提升。

对于 CPP 生产而言，尽管其重构时间对整体加工流程存在较大影响，但仍服从制造车间总完工时间最短的目标，较短的制造时间表明算法性能优越。本节在六个算例维度上比较了所有基线算法和 KGDRL 的调度结果，如下图所示。与基线相比，我们提出的 KGDRL 取得了更好的调度结果。在所有的启发式规则中，MWKR 有着最优的调度结果，而 KGDRL 在五个实例中的表现明显优于 MWKR，这表明 KGDRL 算法在解决 CPP 等高维动态调度问题时的价值。此外，DDQN 得到的结果具有较大的波动性，尽管 DDQN 算法在 5 个机器上的算例表现良好，尤其是在 20*5 的算例上有着明显的提升，但在机器数为 10 的高维算例情况下，它的表现明显低于 MWKR 规则，甚至要低于 FIFO 规则，由此体现出 DDQN 算法学习到的调度策略并不能适用于动态高维场景中。但与 DDQN 算法相比，KGDRL 还提供了更稳定收敛的训练策略，在每个算例下的表现趋向于稳定，并且在大多数情况有着更好的表现。表现出足够的鲁棒性。通过不同算例维度基线算法的表现可以看出，KGDRL 学习到的调度策略具有参数稳定性，其收敛策略能够应用到不同复杂度的算例场景中，体现出其良好的延展性。



机器数为 5 的算例调度结果



机器数为 10 的算例调度结果

参考文献

- [1] Chen X, Huang C, Yao L, et al. Knowledge-guided deep reinforcement learning for interactive recommendation[C]//2020 International Joint Conference on Neural Networks (IJCNN). IEEE, 2020: 1-8.
- [2] Kuhlmann G, Stone P, Mooney R, et al. Guiding a reinforcement learner with natural language advice: Initial results in RoboCup soccer[C]//The AAAI-2004 workshop on supervisory control of learning and adaptive systems. 2004: 2468-2470.
- [3] Yu Y, Zhang P, Zhao K, et al. Accelerating deep reinforcement learning via knowledge-guided policy network[J]. Autonomous Agents and Multi-Agent Systems, 2023, 37(1): 17.
- [4] Sels V, Gheysen N, Vanhoucke M. A comparison of priority rules for the job shop scheduling problem under different flow time-and tardiness-related objective functions[J]. International Journal of Production Research, 2012, 50(15): 4255-4270.
- [5] Gabel T, Riedmiller M. On a successful application of multi-agent reinforcement learning to operations research benchmarks[C]//2007 IEEE international symposium on approximate dynamic programming and reinforcement learning. IEEE, 2007: 68-75.
- [6] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double q-learning[C]//Proceedings of the AAAI conference on artificial intelligence. 2016, 30(1).
- [7] Mnih V, Badia A P, Mirza M, et al. Asynchronous methods for deep reinforcement learning[C]//International conference on machine learning. PmLR, 2016: 1928-1937.
- [8] Silver D, Lever G, Heess N, et al. Deterministic policy gradient algorithms[C]//International conference on machine learning. Pmlr, 2014: 387-395.